

C# PROGRAMMING

(330)

STATE 2022

Production:

Program 1: World Capital Quiz Creator _____ (540 points)

TOTAL POINTS _____ (540 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

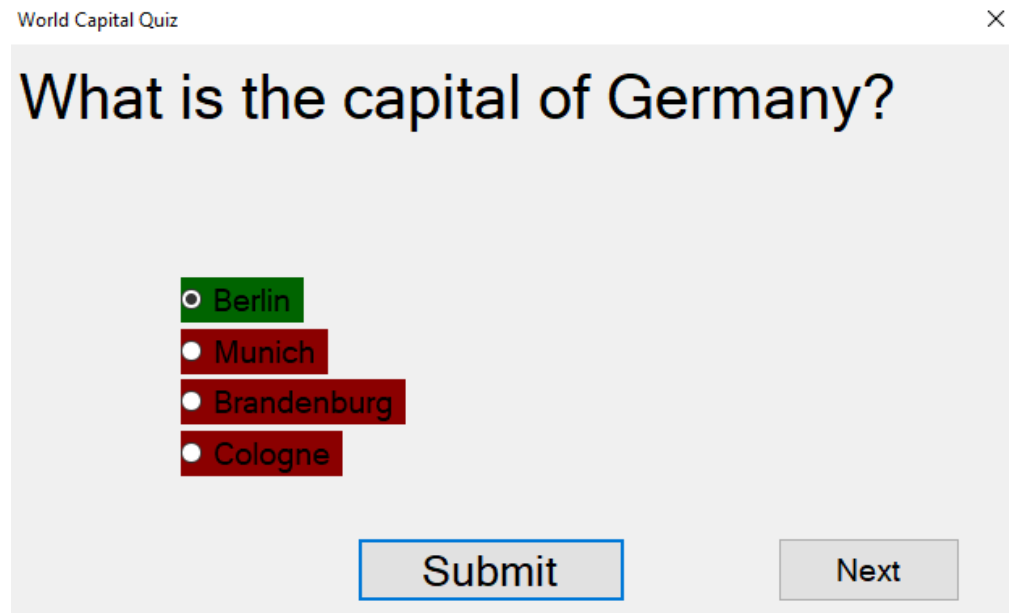
Failure to adhere to any of the following rules will result in disqualification:

1. Contestant must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

World Capital Quiz Creator

In this exercise you will create a C# Windows Form Application that generates a multiple-choice quiz by selecting a certain number of questions and answers from a text file. While the number of questions selected will stay the same for each game, the questions should be randomly selected from the list in the text file every time the game is run.

When started, the program will display the first question in its randomly selected list. Once the user has clicked the radio button corresponding to their selected answer and clicked the “Submit” button, the answers are highlighted (green = the single correct answer, red = all other incorrect answers) and the user can click the “Next” button to advance to the next question. After the user has submitted their answer for the last question and clicked “Next”, a dialog box appears telling the user their score and asking them whether or not they wish to play again.



The screenshot shows a Windows application window titled "World Capital Quiz" with a close button (X) in the top right corner. The main content area displays the question "What is the capital of Germany?". Below the question, there are four radio button options: "Berlin", "Munich", "Brandenburg", and "Cologne". The "Berlin" option is selected, and its label is highlighted in green. The other three options are not selected, and their labels are highlighted in red. At the bottom of the window, there are two buttons: "Submit" and "Next".

Requirements:

1. You must create a C# Windows Form Application named CS _330_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CS_123_45_6789_0000.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. The main form must not have minimize or maximize buttons in the Form Handle.
4. The main form must have text “World Capital Quiz” for its title. The form must start in the center of the screen and must have a Fixed Dialog setting so the size cannot be adjusted.
5. The main form must contain the following items:
 - a. The textbox txtQuestion should also have these additional properties:
 - i. Multiline: true
 - ii. Readonly: true

UI Object	Name	Text	Font Size	Size, Width	Size, Height
Textbox	txtQuestion		29pt	590px	97px
Radio Button	rdoAnswer1		14pt		
Radio Button	rdoAnswer2		14pt		
Radio Button	rdoAnswer3		14pt		
Radio Button	rdoAnswer4		14pt		
Button	btnSubmit	Submit	20pt	166px	40px
Button	btnNext	Next	14pt	113px	40px

- iii.
6. When the program first loads, the main window needs the following (see pictures for reference):
 - a. The Question text box should be place in the top left corner of the window.
 - b. The answer radio buttons should be also be left-aligned
 - c. the Submit button should be center-aligned at the bottom of the window
 - d. the Next button should be right-aligned and also at the bottom of the window.
7. When the program first loads, the quiz questions and answers should be read from a text file named “quizquestion.txt”
 - a. Each line of the text file should represent a different question and its answers. Every question must have exactly four possible answers and only one answer can be considered correct. For best results it is recommended to just use the example question provided in the picture of the example text file. Each line should be formatted as follows (see pictures for example text file):

<QuestionText>,<Answer1>,<Answer2>,<Answer3>,<Answer4>,<CorrectAnswerNumber>

- i. If a line in the text file is not formatted correctly, display a MessageBox with an error message and exit the application (see pictures section for reference)
- ii. If the correct answer number in a line is not a number, display a MessageBox with an error message and exit the application (see pictures section for reference)
- iii. If the correct answer number in a line is not between 1-4, display a MessageBox with an error message and exit the application (see pictures section for reference)

Question
- QuestionText: string - Answers: IList<string> - CorrectAnswerIndex: int
- Question(string inputFile)

- b. Each line of the text file should be read and parsed as an object of a class called “Question”. The class must contain all the information related to a question, including the question text, the possible answers, and which answer is correct. The class must also contain a constructor which takes a line from quizquestion.txt, parses out all question/answer information, and stores it in the appropriate properties.
 - c. From the list of newly created Question objects, randomly select three question to use in the quiz. Finally, load the first question by replacing the question text and answer radio button text with the data for the first question.
8. The “Submit” button must use a click event called “SubmitButton_Click” which performs the following functions:
- a. Retrieves the user-selected answer text as specified by the user-selected radio button
 - i. If none of the answer radio buttons are selected, nothing should happen when the user clicks the Submit button
 - b. Change the BackColor of the answer radio button to reveal the correct and incorrect answers. For the correct answer, use Color.DarkGreen. For the incorrect answers, use Color.DarkRed

9. The “Next button must user a click event called “NextButton_Click” which performs the following functions:
 - a. If the user has answered the current question and there are more questions left in the quiz, load the next question. This can be done by replacing the question text and answer radio button text with the data for the next question. The BackColors for each radio button should also be reset to Color.Transparent.
 - i. If the user hasn’t submitted their answer to the current question, do nothing
 - b. If the user has submitted their answer and the current question is the last one in the quiz, end the quiz. This can be done by showing two dialog boxes, the first telling the user their score and the second asking the user if they want to play again (see pictures for reference).
 - i. If the user selects “Yes”, pull three random question from the list and restart the game.
 - ii. If the user selects “No”, exit the application.

Sample Pictures:

World Capital Quiz ×

What is the capital of Portugal?

- ☐ Porto
- ☐ Vila Nova de Gaia
- ☐ Funchal
- ☐ Lisbon

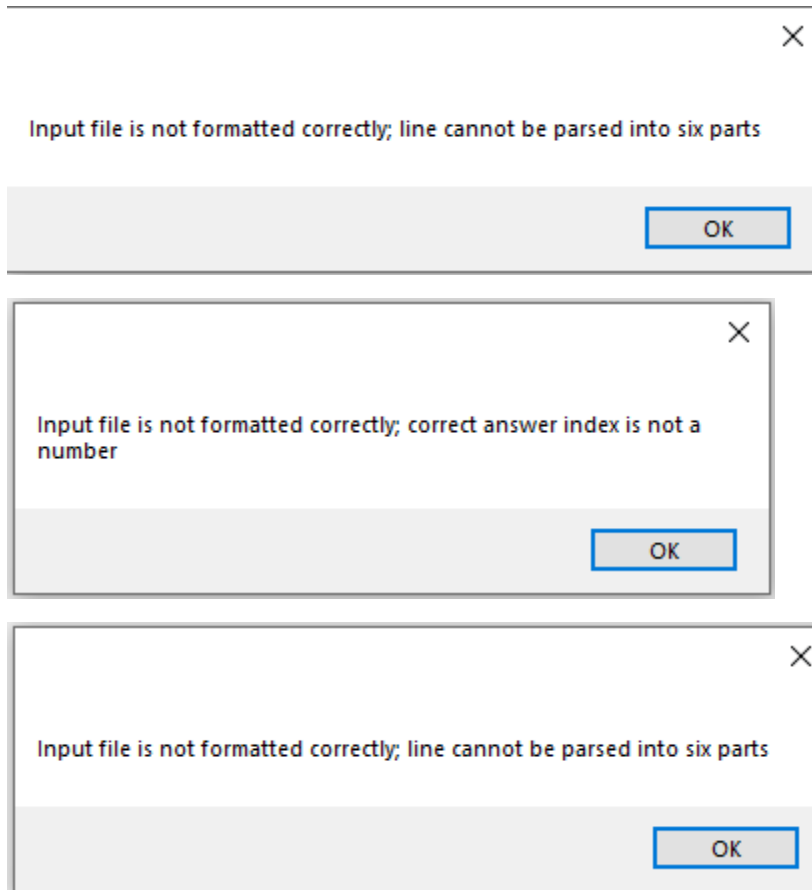
Submit

Next

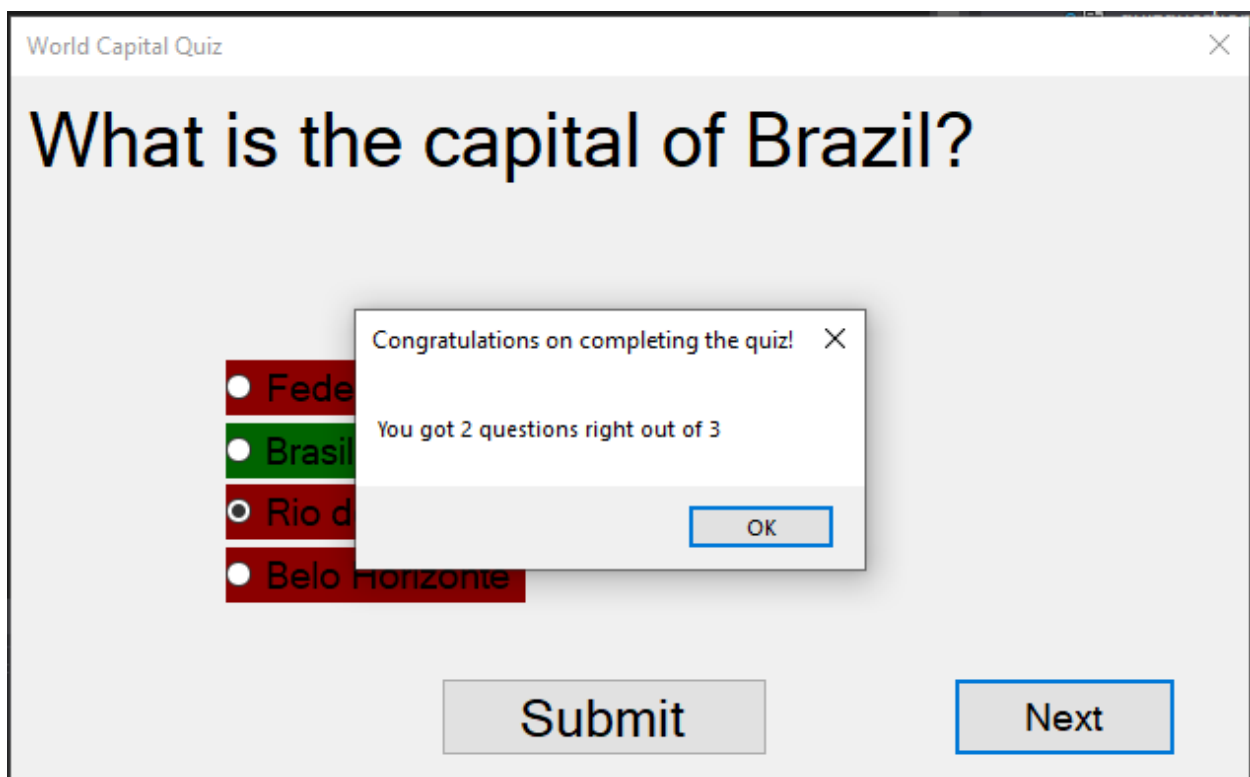
1: Example of the initial layout of the window

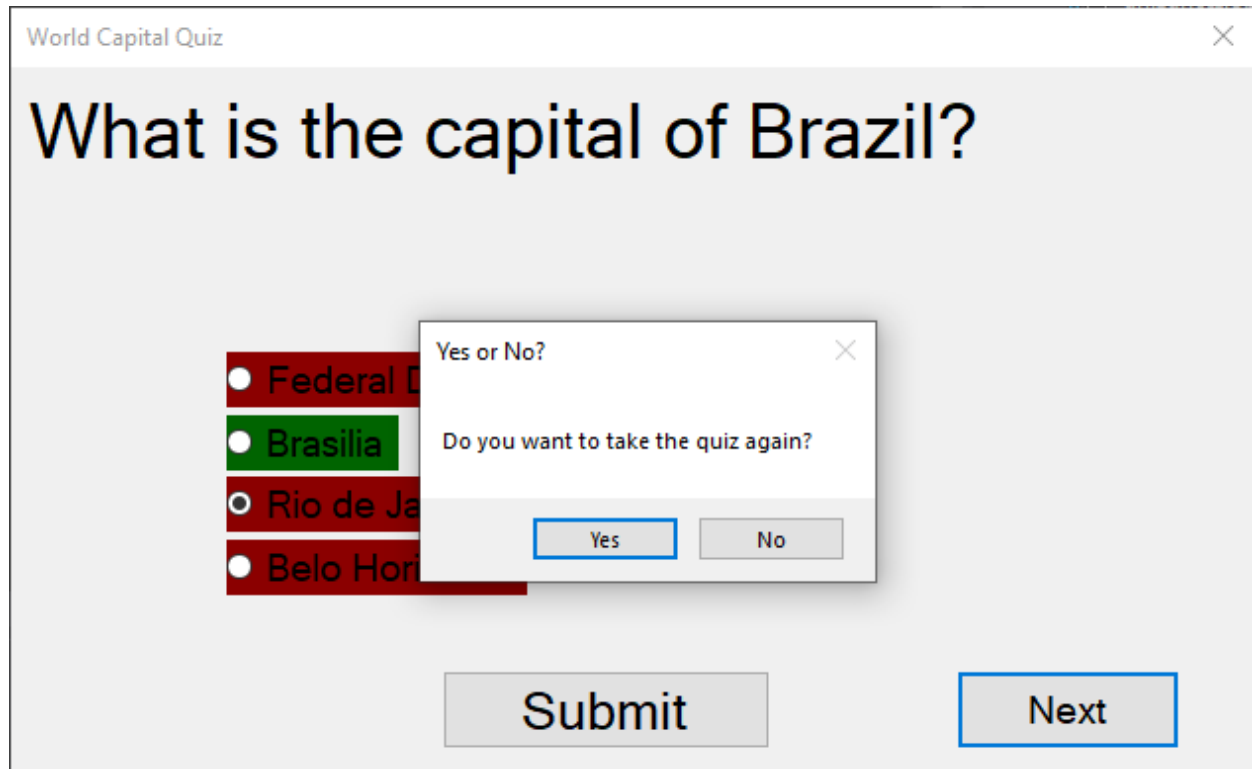
```
What is the capital of Germany?,Berlin,Munich,Brandenburg,Cologne,1
What is the capital of China?,Hangzhou,Yichang,Beijing,Nanning,3
What is the capital of Portugal?,Porto,Vila Nova de Gaia,Funchal,Lisbon,4
What is the capital of New Zealand?,Wellington,Auckland,Christchurch,Melbourne,1
What is the capital of Brazil?,Federal District,Brasilia,Rio de Janeiro,Belo Horizonte,2
What is the capital of Russia?,Saint Petersburg,Moscow,Novosibirsk,Kazan,2
```

2: Example of the quizquestions.txt file



3: Examples of the error message displayed for invalid text file errors





3: Examples of the message boxes displayed when the quiz ends

You will have ninety (90) minutes to complete your work.

Your name and/or school name should not appear on any work you submit for grading.

Remember to use your contestant # where appropriate.

Submit a copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code and/or build and execute your solution/project. **Submissions that do not contain source code will not be graded.**

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).

Your application will be scored on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts

The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to USB drive and program runs from USB _____ 20 pts

If the program does not execute, then the remaining items in the program executive section receive a score of zero.

The title of the form is “World Capital Quiz “ _____ 10 pts

The main form does not have minimize or maximize buttons
in the Form Handle _____ 10 pts

The form’s start position is the center of the screen and it cannot be resized _____ 10 pts

The window has the correct buttons, text boxes, radio buttons and
labels based upon the Figures _____ 30 pts

The program reads all lines from the quizquestions.txt file _____ 50 pts

The program parses each line as a Question object _____ 50 pts

The program randomly selects three questions when starting a quiz	_____ 50 pts
The program correctly loads the first question upon program start	_____ 30 pts
Clicking “Submit” correctly highlights answer text if user answered	_____ 20 pts
Clicking “Submit” does nothing if user has not answered	_____ 10 pts
Clicking “Next” correctly loads the next question	_____ 20 pts
Clicking “Next” resets answer background color to Color.Transparent	_____ 10 pts
Clicking “Next” does nothing if user hasn’t submitted answer	_____ 10 pts
Clicking “Next” on last question displays ending Message Boxes	_____ 20 pts
First ending Message Box gives user their quiz score	_____ 20 pts
Second ending Message Box restarts quiz if “Yes” is selected	_____ 10 pts
Second ending Message Box ends quiz if “No” is selected	_____ 10 pts
The program selects three new random questions upon restart	_____ 30 pts

Source Code Review

Code is commented at the top, for each function, and as needed	_____ 10 pts
Code uses reasonable and consistent variable naming conventions	_____ 10 pts
Code for “SubmitButton_Click” method is present	_____ 30 pts
Code for “NextButton_Click” method is present	_____ 30 pts
Code for reading questions and answers from text file is present	_____ 10 pts
Code for error handling text file parsing errors is present	_____ 20 pts
Code for parsing each text file line as a Question object is present	_____ 10 pts
Code for defining Question class is present	_____ 10 pts
Code for randomly selecting three questions for quiz is present	_____ 10 pts
Code for ending the quiz correctly is present	_____ 10 pts

Total Points:

_____ / 540 pts

Potential Solution

```
// CS_330_01_2345_6789
using CSharpProgramming_State.Properties;
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Windows.Forms;

namespace CSharpProgramming_State
{
    public partial class QuizShowForm : Form
    {
        private IList<Question> questions;
        private IList<RadioButton> answerButtons;
        private int currentQuestion = 0;
        private bool hasAnswered = false;
        private int score = 0;
        private Random random;

        private readonly int NumberOfQuestions = 3;

        public QuizShowForm()
        {
            InitializeComponent();
            random = new Random();
            questions = getRandomQuestions(NumberOfQuestions);
            answerButtons = new List<RadioButton> { Answer1Radio, Answer2Radio,
            Answer3Radio, Answer4Radio };
            LoadQuestion(questions.First());
        }

        private IList<Question> getRandomQuestions(int numberOfQuestions)
        {
            var questionsFromFile = readQuestionsFromFile();
            var randomizedQuestions = new List<Question>();
            for (var i = questionsFromFile.Count; i > 0; i--)
            {
                var randomIndex = random.Next(0, i);
                randomizedQuestions.Add(questionsFromFile[randomIndex]);
                questionsFromFile.RemoveAt(randomIndex);
            }
            return randomizedQuestions.Take(numberOfQuestions).ToList();
        }

        private IList<Question> readQuestionsFromFile()
        {
            var questions = new List<Question>();
            var questionsAndAnswers = Resources.quizquestions.Split(new[] {
            Environment.NewLine }, StringSplitOptions.RemoveEmptyEntries).ToList();
            questionsAndAnswers.RemoveAt(0);
            foreach (var qna in questionsAndAnswers)
            {
                var parts = qna.Split(',');

                if (parts.Length != 6)
                {

```

```

        MessageBox.Show("Input file is not formatted correctly; line cannot
be parsed into six parts");
        Application.Exit();
    }

    if (!int.TryParse(parts[5], out int correctAnswer))
    {
        MessageBox.Show("Input file is not formatted correctly; correct
answer index is not a number");
        Application.Exit();
    }

    var answers = new List<string> { parts[1], parts[2], parts[3], parts[4]
};

    if (correctAnswer < 1 || correctAnswer > answers.Count)
    {
        MessageBox.Show("Input file is not formatted correctly; correct
answer index does not represent a valid answer position");
        Application.Exit();
    }

    var question = new Question
    {
        QuestionText = parts[0],
        Answers = answers,
        CorrectAnswerIndex = correctAnswer - 1
    };
    questions.Add(question);
}
return questions;
}

private void LoadQuestion(Question question)
{
    resetAnswers();
    QuestionTextbox.Text = question.QuestionText;
    Answer1Radio.Text = question.Answers[0];
    Answer2Radio.Text = question.Answers[1];
    Answer3Radio.Text = question.Answers[2];
    Answer4Radio.Text = question.Answers[3];
}

private void resetAnswers()
{
    foreach (var answer in answerButtons)
    {
        answer.BackColor = Color.Transparent;
        answer.Checked = false;
    }
}

private void NextButton_Click(object sender, EventArgs e)
{
    if (!hasAnswered) return;
    if (currentQuestion < questions.Count - 1)
    {
        var nextQuestion = questions[++currentQuestion];
        LoadQuestion(nextQuestion);
    }
    else

```

```

        {
            MessageBox.Show($"You got {score} questions right out of
{questions.Count}", "Congratulations on completing the quiz!");
            var dialogResponse = MessageBox.Show("Do you want to take the quiz
again?", "Yes or No?", MessageBoxButtons.YesNo);
            if (dialogResponse == DialogResult.Yes)
            {
                questions = getRandomQuestions(NumberOfQuestions);
                LoadQuestion(questions.First());
                currentQuestion = 0;
                score = 0;
            }
            else
            {
                Application.Exit();
            }
        }
        hasAnswered = false;
    }

    private void SubmitButton_Click(object sender, EventArgs e)
    {
        var current = questions[currentQuestion];
        for (var i = 0; i < current.Answers.Count; i++)
        {
            if (i == current.CorrectAnswerIndex)
            {
                answerButtons[i].BackColor = Color.DarkGreen;
                if (answerButtons[i].Checked)
                {
                    score++;
                }
            }
            else
            {
                answerButtons[i].BackColor = Color.DarkRed;
            }
        }
        hasAnswered = true;
    }
}

public class Question
{
    public string QuestionText { get; set; }
    public IList<string> Answers { get; set; }
    public int CorrectAnswerIndex { get; set; }
}

```